Towards On-Wearable Human Activity Recognition Deep Learning with Learnable Quantization

QUANG THONG PHAN, University of Siegen, Germany
KRISTOF VAN LAERHOVEN, University of Siegen, Germany

Human Activity Recognition (HAR) on wearable devices is significantly constrained by limited battery capacity, processing resources, and memory bandwidth. Traditional HAR pipelines exacerbate these issues by relying on high-precision floating-point representations, which demand substantial data throughput and energy consumption. To address these challenges, we investigate the use of non-linear, learnable quantization for efficient HAR on low-power wearables. Building upon the existing γ-QUANT framework—which suffers from high computational overhead due to power function emulation—we propose a hardware-friendly alternative termed Softsign-QUANT.

We systematically evaluate the impact of this approach on predictive performance and hardware efficiency using benchmark HAR datasets. Our results demonstrate that aggressively compressing input data to 4-bit representations preserves essential, activity-relevant information, achieving comparable macro F1-scores to both the uncompressed baselines and γ-QUANT. Furthermore, we examine design trade-offs between shared and per-axis quantization parameters, finding that shared, shared-axis parameters are largely sufficient for general HAR tasks, effectively minimizing memory overhead.

To bridge theoretical design and physical deployment, we integrate the learnable quantization layer into an integer-only inference pipeline on an open-source smartwatch based on an ARM Cortex-M4 microcontroller. Hardware profiling reveals that while downstream inference latency and memory footprints remain identical to the baseline, the proposed Softsign-QUANT achieves net-positive system energy conservation. By optimizing data movement and utilizing hardware-supported mathematical operations, Softsign requires only 28 CPU cycles per sample compared to the 153 cycles demanded by γ-QUANT. Our findings validate learnable 4-bit Softsign quantization as a practical, energy-efficient mechanism for deploying deep learning-based HAR on memory-constrained edge architectures.

CCS Concepts: • **Human-centered computing → Ubiquitous and mobile computing design and evaluation methods**; • **Computing methodologies → Neural networks**.

Additional Key Words and Phrases: wearable activity recognition, inertial-based activity recognition, human activity recognition, learnable quantization

ACM Reference Format:
Quang Thong Phan and Kristof Van Laerhoven. 2026. Towards On-Wearable Human Activity Recognition Deep Learning with Learnable Quantization. *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 1, 1 (February 2026), 18 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Human Activity Recognition (HAR) on wearable devices faces significant challenges due to the inherent constraints of limited battery capacity, restricted processing resources, and narrow memory bandwidth. These limitations are particularly acute in always-on, battery-powered wearables, where energy efficiency is as critical as classification

Authors' Contact Information: Quang Thong Phan, quang.phan@student.uni-siegen.de, University of Siegen, Siegen, Germany; Kristof Van Laerhoven, kvl@eti.uni-siegen.de, University of Siegen, Siegen, Germany.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 2474-9567/2026/2-ART
https://doi.org/XXXXXXX.XXXXXXX

accuracy. Traditional HAR pipelines exacerbate these constraints by relying on high-precision, normalized floating-point data generated from high-bit Analog-to-Digital Converters (ADCs). This approach results in large data throughputs that demand substantial computational overhead, both for feature extraction and inference, making it poorly suited for the energy-constrained environments of wearable devices.

The conventional HAR processing pipeline, where raw sensor signals are uniformly sampled at fixed, high rates and converted into 16- or 32-bit floating-point representations, is computationally expensive and inefficient: It assumes a worst-case scenario for all activities, regardless of whether the sensor data is informative or redundant. For instance, periods of inactivity or low-motion contexts (such as sitting or sleeping) generate large volumes of data that contribute little to the recognition task, yet they consume valuable energy and processing resources. This inefficiency is particularly problematic for long-term, continuous monitoring applications, where the wearable's battery life and real-time responsiveness are important.

In response to these challenges, recent work has investigated task-specific, learnable quantization methods to reduce the computational and energy overhead of deep learning–based HAR. One such approach is γ-QUANT [13], which proposes a non-linear, learnable quantization scheme in which quantization parameters are optimized jointly with the downstream neural network. However, the choice of the power function remains a critical challenge when deploying to low-power devices. Moreover, the design choices surrounding such quantization, such as whether to use shared or per-axis quantization parameters, can significantly impact both memory overhead and model performance, yet these trade-offs have not been systematically evaluated in the context of HAR. We focus in this article on several contributions that deepen our understanding of using non-linear, learnable quantization schemes in HAR:

- We investigate the trade-offs between the established γ-QUANT and our proposed Softsign-QUANT. We show that the Softsign approach effectively preserves activity-relevant information, achieving comparable F1-scores to γ-QUANT on benchmark datasets despite its simpler mathematical formulation.
- We bridge the gap between theoretical quantization and physical deployment by integrating the input quantization layer into a full integer-only inference pipeline on an open-source smartwatch.
- We investigate the trade-offs between shared and per-axis quantization, providing evidence that shared parameters are sufficient for HAR tasks, thereby further minimizing memory overhead.

2 Related Work

Research in HAR on wearable devices has attracted interest at the intersection of sensing, embedded systems, and machine learning. Prior work spans hardware-aware system design, efficient neural inference, and algorithmic adaptations aimed at operating under the stringent resource constraints of always-on wearable platforms. This section reviews relevant literature across these dimensions, highlighting how limitations in computation, memory, and energy have shaped current approaches.

Hardware Constraints in Wearable HAR. Unlike smartphones or edge servers, wearable sensing platforms are built around ultra-low-power microcontrollers designed for continuous operation with restricted energy and memory resources [1, 29]. These systems typically operate at clock frequencies up to 200 MHz and provide limited on-chip memory, often constrained to a few hundred kilobytes of SRAM and at most a few megabytes of Flash [10, 28]. Such constraints are further compounded by the reliance on small rechargeable or coin-cell batteries [12, 22, 24], which necessitate strict power management strategies to support always-on sensing and long deployment lifetimes. As a result, wearable HAR systems must carefully balance sensing fidelity, computational load, and energy consumption [11, 31]. Within this context, the traditional HAR pipeline [2, 6, 23, 34], characterized by high-frequency sampling, high-bit-depth ADC, and floating-point signal processing, poses significant challenges. High-precision sensor streams generate substantial data throughput, placing pressure on memory bandwidth and increasing the energy cost of both data movement and computation [25, 32]. Such limitations motivate the need

for system-level optimizations that extend beyond algorithmic accuracy and explicitly account for the constraints of wearable hardware.

Efficient Neural Inference at the Edge. To address such hardware limitations, the field of TinyML has focused on optimizing neural networks for MCU-class devices. Prior efforts have explored a range of model-level optimization techniques, including structured and unstructured network pruning [14, 16, 21, 30], low-bit quantization of weights and activations [7, 19, 35], knowledge distillation [9, 20], and hardware-aware Neural Architecture Search [15, 18, 27]. These techniques aim to reduce model size, computational complexity, and memory access, making deep learning feasible within tight resource budgets. In parallel, several lightweight architectures tailored specifically to HAR have been proposed. Models such as TinyHAR [37], TinierHAR [4] and MLP-HAR [36], demonstrate that careful architectural design can significantly reduce parameter counts and inference latency while preserving recognition performance. Complementing these algorithmic advances, optimized inference runtimes such as TensorFlow Lite for Microcontrollers [8] and AIfES [33] provide highly efficient integer-only kernels and memory planning strategies that enable deployment on severely constrained devices. These approaches primarily focus on optimizing the neural network architecture and its execution, assuming that sensor data is already available in a fixed, high-precision digital form.

Input Quantization and Adaptive Sensing. A smaller but growing line of research has begun to question this assumption by examining the role of input representations in resource-constrained learning systems. Learnable quantization methods, including non-linear and task-aware schemes, have shown promise in reducing numerical precision while preserving task-relevant information. Recent work such as γ-QUANT [13] demonstrates that quantization parameters can be optimized jointly with the downstream model, enabling aggressive compression of input signals with minimal impact on accuracy.

Modern Inertial Measurement Units (IMUs) are increasingly designed as highly integrated System-on-Chip devices that offer flexible configuration and on-chip processing capabilities [3, 38], making it feasible to implement parts of an adaptive quantization strategy directly within the sensor hardware. For example, the STMicroelectronics LSM6DSV32XTR[1] includes programmable finite state machines and a machine learning core that allow motion tracking and context-aware processing to be executed within the sensor, reducing the need for high-rate data transfer to the host processor and enabling real-time reconfiguration of sampling behavior based on detected motion patterns. Similarly, the Bosch Sensortec BHI IMUs [2] are optimized for wearable applications with on-chip activity and context recognition features that can autonomously signal significant motion events and generate interrupts without waking the main system CPU, effectively reducing unnecessary sampling and data flow. These sensors also expose configurable output data rates, interrupt generation, and FIFO buffering, which can be exploited to implement dynamic reporting and resolution adjustment policies at the sensor level. By mapping learnable quantization and adaptive sampling strategies directly to such on-chip features, systems could reduce memory bandwidth, lower energy consumption due to fewer high-precision data transfers, and enable more efficient, task-aware sensing that aligns closely with the actual dynamics of human activity signals.

While existing approaches successfully optimize the model parameters and inference engine, they typically overlook the efficiency of the data acquisition stage. Standard pipelines still utilize high-bit depth sensor data (e.g., 12-, 16-, or even 32-bit) at the input, incurring significant memory bandwidth and energy costs before the data even reaches the neural network. In contrast, our work explores a novel direction by optimizing the quantization of the input signal itself. We propose a learnable, non-linear quantization scheme that operates directly at the ingestion stage, effectively acting as a task-specific ADC configuration to further minimize system overhead beyond what is possible with model compression alone.

[1]Datasheet: https://www.st.com/en/mems-and-sensors/lsm6dsv32x.html [last access February 2026]
[2]Datasheet: https://www.bosch-sensortec.com/products/smart-sensor-systems/ [last access February 2026]

3 Input Quantization

3.1 Proposed Framework

HAR data from IMUs often exhibits a highly non-uniform distribution, with most informative samples concentrated near zero acceleration due to subtle, continuous movements, and with large-magnitude values occurring infrequently during high-intensity actions. This imbalance means that fine-grained essential information to activities is primarily contained in low-magnitude regions of the signal. To exploit this statistical property, we adopt the framework, which models quantization as a learnable non-linear transformation, rather than using fixed, uniform levels. This approach allocates higher precision to zero-centered regions where critical motion details reside, and coarser resolution to the rare, high-magnitude outliers (Figure 1). By jointly optimizing quantization with downstream learning objectives, $Softsign$ quantization preserves essential motion features while enabling efficient compression:

$$Q(\mathcal{X}, k, \mu) = \text{round}\left(\frac{k(\mathcal{X} - \mu)}{1 + |k(\mathcal{X} - \mu)|} \cdot (2^b - 1)\right)$$

where $\mathbf{X} \in \mathbb{R}^L$ denotes a window of normalized accelerometer inputs containing values $x_i \in [-1, 1]$, with k controlling the curvature of the quantization steps, μ being a learnable offset that determines the degree to which quantization levels are concentrated around zero, and b is the target bit-depth of the quantized signal.

images/quantize.png

Fig. 1. Impact of learnable parameters k and μ on quantization resolution. The top row compares different transformation behaviors. The bottom row illustrates the showing how the raw Laplacian-like accelerometer input is transformed and mapped into discrete 4-bit bins, effectively preserving information in the low-magnitude region.

3.2 Theoretical Efficiency

To evaluate the energy efficiency of the proposed method, we model a typical data processing workflow utilizing a low power MCU coupled with a modern IMU. Our comparative analysis evaluates two distinct processing pipelines: a standard high-bit baseline workflow, and the proposed quantized low-bit methodology.

The overall energy usage is dictated by this continuous execution pipeline, which comprises several sequential stages from data sampling to model inference. In this architecture, the host MCU, frequently an ARM Cortex-M4,

images/theoretical_efficiency.png

Fig. 2. Theoretical efficiency modeling of the typical high-bit data processing workflow compared with the proposed low-bit quantization pipeline. The total energy is decomposed into sequential stages: data sampling ($E_{sampling}$), quantization from high-bit to low-bit ($E_{quantization}$), packing samples into byte-aligned registers ($E_{packing}$), transferring data from the sensor SRAM to the FIFO buffer ($E_{move(sensor)}$), host MCU wake-up (E_{wake}), copying data from the FIFO buffer to the MCU SRAM ($E_{move(mcu)}$), unpacking the payload ($E_{unpacking}$), and executing the final model inference ($E_{inference}$).

interfaces with the sensor node via a standard SPI/I2C connection. Our model specifically targets the BHI360's integrated sensor core utilizing the ARC EM4 architecture [3]. This allows our theoretical energy calculations to closely reflect the realistic execution of in-sensor machine learning workloads.

To formally evaluate the energy efficiency of the proposed methodology, we define a theoretical energy consumption model based on the CPU logic cycles required across each stage of the data processing pipeline. Let N denote the total number of samples for 1D sensor, defined as $N = f_s \cdot T$, where f_s is the sampling rate and T is the sampling duration. Because modern microcontrollers process data in byte-aligned formats, the total data volume V required to accommodate these samples in 8-bit registers is contingent upon the selected bit-depth b, where $b \in \{4, 12, 16, 32\}$. This data volume is calculated as:

$$V = \lceil N \cdot b/8 \rceil$$

Additionally, let e_{cyc} represent the baseline energy consumed per logic cycle, and let S denote the capacity of the FIFO buffer for 1D sensor in bytes. Because explicit power consumption metrics for specific in-sensor processing units are often unpublished in manufacturer datasheets , our theoretical efficiency modeling operates under the baseline assumption that the energy per logic cycle (e_{cyc}) for the ARC EM4 core is comparable to that of the host ARM Cortex-M4 architecture. The net energy savings (ΔE) provided by the proposed method are quantified by calculating the difference in total energy usage between the standard high-bit pipeline and the low-bit pipeline across all sequential stages.

$$\Delta E = \Delta E_{sampling} + \Delta E_{quant} + \Delta E_{packing} + \Delta E_{move(sensor)} + \Delta E_{wake} + \Delta E_{move(mcu)} + \Delta E_{unpacking} + \Delta E_{inference}$$

Notably, there are no difference in the energy costs for both the sampling stage ($\Delta E_{sampling} = 0$) and the model inference stage ($\Delta E_{inference} = 0$) between the two approaches. This parity exists because both pipelines process the identical number of initial samples. Furthermore, TensorFlow Lite framework require inputs to be converted into an 8-bit integer (int8) format prior to inference; thus, both the high-bit and low-bit pipelines must ultimately

[3]Datasheet: https://www.bosch-sensortec.com/products/smart-sensor-systems/bhi360/ [last access February 2026]

align to this format before feeding data into the ML model on the MCU. detailed breakdown of the remaining energy differentials is presented in Table 1.

Table 1. Theoretical energy cost formulas for the continuous execution pipeline stages. C denotes the number of CPU cycles required to perform a given operation.

Processing stages	$E_{stage}^{high-bit} - E_{stage}^{low-bit}$
$\Delta E_{\text{quantization}}$	$0 - C_{\text{quant}} \cdot e_{\text{cyc}}$
$\Delta E_{\text{packing}}$	$(N/2) \cdot (C_{\text{packing}}^{\text{high-bit}} - C_{\text{packing}}^{\text{low-bit}}) \cdot e_{\text{cyc}}$
$\Delta E_{\text{move(sensor)}}$	$(V^{high-bit} - V^{low-bit}) \cdot C_{\text{move(sensor)}} \cdot e_{\text{cyc}}$
ΔE_{wake}	$(\lceil V^{high-bit}/S \rceil - \lceil V^{low-bit}/S \rceil) \cdot C_{\text{wake}} \cdot e_{\text{cyc}}$
$\Delta E_{\text{move(mcu)}}$	$(V^{high-bit} - V^{low-bit}) \cdot C_{\text{move(mcu)}} \cdot e_{\text{cyc}}$
$\Delta E_{\text{unpacking}}$	$(N/2) \cdot (C_{\text{unpacking}}^{\text{high-bit}} - C_{\text{unpacking}}^{\text{low-bit}}) \cdot e_{\text{cyc}}$

4 Implementations

In the following section, we validate our approach by detailing the experimental framework, including the selected benchmark dataset, the lightweight network architecture, and the specific evaluation steps.

4.1 Datasets

To validate our quantization approach, we employ two distinct datasets. First, we utilize the WEAR dataset [5], which provides tri-axial accelerometer data sampled at 50 Hz from 18 subjects performing 18 body-weight exercises in an outdoor environment. To emulate the constraints of a single-sensor smartwatch, we isolate the inertial stream from the left wrist and aggregate the original fine-grained activities into eight distinct super-classes (e.g., consolidating five variants of jogging into a single 'jogging' class). Additionally, we also use the UCI-HAR dataset [26], which also features tri-axial data sampled at 50 Hz. This dataset comprises recordings from 30 volunteers (aged 19–48) performing six activities of daily living. Details are described in Table 2.

Table 2. Overview of the datasets and sliding window configurations employed in the study.

Dataset	**Classes**	**Window Size (s)**	**Window Size (N)**	**Overlap**
WEAR	8	2.00	100	50%
UCI-HAR	6	2.56	128	50%

4.2 Experimental Setup

We investigate two parameterization strategies for the low-bit quantization of IMU input data. First, we evaluate a shared-axis approach, where a single pair of parameters (k, μ) is learned across the entire dataset. Second, we examine a per-axis quantization strategy, in which independent parameters (k, μ) are learned for each individual sensor axis. In both cases, the data is quantized to a 4-bit ($b = 4$) representation. These low-bit pipelines are

then compared against a standard 12-bit ($b = 12$) baseline, which is representative of typical Analog-to-Digital Converter (ADC) resolutions.

Input signals are first normalized to the range $[-1, 1]$ via min-max normalization applied globally across the full dataset. The model is trained using the Adam optimizer with a learning rate of $1e^{-4}$ and weight decay of $1e^{-6}$. We employ weighted cross-entropy loss and batch size of 64. To assess the robustness of γ-QUANT against inter-subject variability, we conduct a Leave-One-Subject-Out (LOSO) cross-validation scheme, reporting the class-averaged macro F1-score and accuracy averaged across all validation splits.

We evaluate the framework's suitability for resource-constrained hardware by deploying a lightweight Convolutional Neural Network (CNN) architecture explicitly optimized for 1D inertial time-series data (Figure 3). Specfically, Separable Convolutions is utilized to decouple spatial and depthwise feature learning. This design choice drastically reduces model complexity, resulting in a highly compact architecture with only 37,955 trainable parameters. We then introduce the quantization layer $Q_{k,\mu}$ as a learnable pre-processing stage. Crucially, this layer is inserted between the input layer and the reshaping operation. This placement allows the model to learn the optimal non-linear quantization directly on the raw signal stream, effectively simulating the behavior of a task-specific ADC before the data is reconstructed into sensor channels. Mathematically, the forward pass for the proposed architecture is defined as:

$$\hat{\mathbf{y}} = f_{\text{CNN}}\left(\text{Reshape}\left(Q_{k,\mu}(x)\right)\right)$$

where f_{CNN} represents the lightweight architecture described, Q is the learnable quantization layer. Because the discrete nature of the quantization operation inherently halts the flow of gradients during the backward pass, we employ a Straight-Through Estimator (STE) to allow gradient-based optimization of k and μ.

images/model_arch.png

Fig. 3. Architecture Overview. The Baseline CNN processes raw floating-point data reshaped into tri-axial windows. The proposed $Softsign$ quantization integration inserts a learnable quantization layer directly on the flattened input stream before structural reshaping, enabling the optimization of k and μ jointly with the downstream classifier.

Upon the completion of training, the optimized quantization parameters k and μ are extracted. The learnable quantization module is then implemented directly on the IMU processing core to quantize the input representations during inference, as detailed in Appendix 7. The remainder of the neural network is deployed on the ARM Cortex-M4 MCU host as stated in Figure 4. As required by the TensorFlow Lite framework, the model's weights and biases are subjected to post-training quantization into 8-bit integers. Following this conversion, the model is serialized into the TensorFlow Lite FlatBuffer format to leverage the highly optimized execution kernels provided by TensorFlow Lite for Microcontrollers [8]. We characterize system efficiency through deterministic hardware profiling instead of physical instrumentation to ensure reproducibility. Specifically, Flash usage is reported as

the static binary size of the compiled firmware, including the inference engine and DSP blocks. RAM utilization denotes the peak runtime memory requirement, comprising both static allocations and the dynamic tensor arena. Inference latency is estimated via cycle-accurate profiling calibrated for the Cortex-M4 at 64 MHz; we report the mean execution time per sample (ms) averaged over the test set to account for data-dependent variability.

images/dataflow.png

Fig. 4. Dataflow of the proposed quantization pipeline, illustrating the division of computational tasks between the in-sensor fuser core and the host MCU.

5 Evaluation results

The subsequent section details our experimental findings, providing a comprehensive comparison between the proposed Softsign-QUANT and the recent γ-QUANT method across two key metrics: model performance and energy efficiency.

5.1 Performance

The most significant finding from our evaluation is the minimal degradation in predictive performance across both datasets, despite the aggressive reduction in input bit-depth from standard 12-bit floats to 4-bit input representations.

As detailed in Table 3, the γ-QUANT method exhibits high resilience to compression. On the WEAR dataset, the shared-axis γ-QUANT maintains a float32 macro F1-score of 69.52%, representing a negligible drop of only 1.02% compared to the uncompressed baseline of 70.54%. This trend holds on the less complex UCI-HAR dataset, where the per-axis γ-model achieves an F32 macro F1-score of 88.04%, closely tracking the 89.60% baseline. This effectively demonstrates that the task-relevant structural information in HAR signals is preserved during extreme low-bit compression. While our propsoed hardware-friendly Softsign-QUANT trades slightly lower expressivity for computational efficiency, it remains highly competitive. On the WEAR dataset, the Softsign per-axis F32 model achieves a macro F1-score of 68.48%. Interestingly, on the UCI-HAR dataset, the Softsign per-axis approach (88.50%) slightly outperforms its γ counterpart (88.04%), indicating that hardware-efficient activation functions can serve as highly effective quantization mappings depending on the activity characteristics.

To further assess the robustness of the quantized models against inter-subject variability, we examined the subject-specific performance boundaries within the LOSO cross-validation setup. By analyzing the int8 accuracy fold distributions for the UCI-HAR dataset, we observed that the per-axis Softsign strategy provides substantially better resilience against outlier subjects compared to the shared-axis approach. Specifically, the Softsign per-axis model yields a peak accuracy of 90.50% (best-case), with its worst-case performance tightly bounded at 81.88% when evaluated on the most challenging outlier subject in the cohort. Conversely, while the shared-axis Softsign approach achieves a comparable best-case accuracy of 90.36%, its performance catastrophically degrades to 70.58% when confronted with that exact same outlier subject. This significant 11.3% improvement in worst-case accuracy demonstrates that learning channel-specific scale and offset parameters allows the model to adapt to unusual, subject-specific movement patterns, ultimately ensuring a much more stable and reliable performance variance across the entire population (±2.36% versus ±4.47%).

Table 3. Performance metrics on WEAR and UCI HAR datasets. (DATA LEAKAGE)

	Quantization Function	Quantization Configuration	Input Representation	Weights and biases data type: int8		Weights and biases data type: float32	
				Accuracy	Macro F1	Accuracy	Macro F1
WEAR	Baseline	N/A	Default (8-bit)	70.74 ± 4.17	69.05 ± 4.16	72.13 ± 4.57	70.54 ± 4.71
	Gamma	Per-axis	4-bit	70.20 ± 3.65	68.20 ± 3.71	71.42 ± 3.93	69.67 ± 4.01
	Gamma	shared-axis	4-bit	70.39 ± 3.71	68.59 ± 3.78	71.41 ± 3.94	69.52 ± 3.96
	Softsign	Per-axis	4-bit	69.76 ± 4.38	67.55 ± 4.37	70.48 ± 4.34	68.48 ± 4.38
	Softsign	shared-axis	4-bit	69.26 ± 4.35	67.10 ± 4.24	70.13 ± 4.70	68.00 ± 4.73
UCI-HAR	Baseline	N/A	Default (8-bit)	89.43 ± 1.22	89.50 ± 1.22	89.54 ± 1.21	89.60 ± 1.21
	Gamma	Per-axis	4-bit	87.77 ± 2.41	87.83 ± 2.44	87.77 ± 2.62	88.04 ± 2.62
	Gamma	shared-axis	4-bit	87.57 ± 1.32	87.57 ± 1.28	87.65 ± 1.16	87.64 ± 1.16
	Softsign	Per-axis	4-bit	88.17 ± 2.36	88.28 ± 2.31	88.39 ± 2.23	88.50 ± 2.22
	Softsign	shared-axis	4-bit	86.58 ± 4.47	86.59 ± 4.39	86.95 ± 4.47	86.94 ± 4.48

Comparing the two proposed parameterization strategies, the per-axis and shared-axis quantizations perform comparably in the generalized LOSO setting. For instance, on the UCI-HAR dataset, Softsign-QUANT yields 88.17% (per-axis) versus 86.58% (shared-axis). While per-axis quantization theoretically offers higher expressivity by modeling sensor-specific distributions, these results indicate that shared, shared-axis parameters are largely sufficient for this application. This is a highly favorable outcome for embedded deployments on platforms like the Cortex-M4, as shared-axis quantization significantly reduces the memory footprint required to store the scaling and offset parameters.

Notably, the choice of quantization method does not impact downstream inference latency. Because modern microcontrollers typically lack native 4-bit arithmetic logic units, the 4-bit quantized inputs are universally processed as standard 8-bit integers during model execution, resulting in identical inference speeds across all configurations, as stated in Table 4.

Table 4. Hardware performance metrics of the baseline model versus the proposed method, profiled for ARM Cortex M4, 64MHz using the Edge Impulse toolchain [17]. Flash and RAM values denote the static binary size and peak runtime memory allocation.

Configuration	Input Representation	Inference Latency (ms)	Peak RAM (kB)	Flash Usage (kB)
Baseline without quantization	8-bit	54.5 ± 4.5	23.9	139
γ-Quant preprocessing	4-bit	54.5 ± 4.5	23.9	139
Softsign-Quant preprocessing	4-bit	54.5 ± 4.5	23.9	139

5.2 Energy Efficiency

To ground our theoretical model, the net system energy savings (ΔE_{sys}) can be calculated by substituting the stage-specific cycle counts, detailed in the Table 7, into the previously defined energy differential equation. The expanded formulation is expressed as follows:

$$\Delta E_{\text{sys}} = \left[\frac{N}{2} \cdot 9 - \frac{N}{2} \cdot 3\right] e_{\text{cyc}} + 2 \cdot [V_{12} - V_4] \cdot 6 \cdot e_{\text{cyc}} + \left(\left\lceil \frac{V_{12}}{S} \right\rceil - \left\lceil \frac{V_4}{S} \right\rceil\right) \cdot 1056 \cdot e_{\text{cyc}} + [V_{12} \cdot 6 - V_4 \cdot 2] \cdot e_{\text{cyc}} - E_{\text{quantization}}$$

The computational penalty introduced by the quantization stage ($E_{\text{quantization}}$) depends strictly on the selected algorithmic approach. Specifically, the hardware-supported Softsign-QUANT requires 28 CPU cycles per sample, whereas the γ-QUANT requires approximately 153 cycles per sample, as detailed in Table 5 and 6. To visualize in a realistic hardware context, we calculate the baseline logic cycle energy (e_{cyc}) utilizing the specifications of the nRF52840 ARM Cortex-M4 MCU[4]. The active power consumption (P_{active}) is derived from a current draw of 3.3 mA operating at 3.0 V, yielding 9.9 mW. For a processor operating at 64 MHz, the theoretical energy expended per logic cycle is calculated as:

$$e_{cyc} = \frac{P_{active}}{f_{cpu}} = \frac{9.9 \text{ mW}}{64 \text{ MHz}} \approx 155 \text{ pJ}$$

images/f_N_plot.png

Fig. 5. Comparison of net energy savings (ΔE) across different quantization methods, the number of samples (N) and FIFO buffer sizes (S) with $e_{cyc} \approx 155$ pJ.

Figure 5 illustrates the net energy savings (ΔE) for 1D sensor data processing as a function of the number of samples (N) and the configurable FIFO buffer capacity (S). Because modern IMUs allow for flexible buffer configurations, we evaluate three representative sizes: $S \in \{50, 100, 150\}$ bytes each 1D sensor. Our analysis yields two primary insights. First, when comparing the two algorithms, the high computational overhead of the

[4]Datasheet: https://docs.nordicsemi.com/bundle/ps_nrf52840/page/keyfeatures_html5.html [last access February 2026]

power function emulation required for γ-QUANT results in a systemic energy deficit on resource-constrained devices. Conversely, the hardware-friendly Softsign method achieves net-positive energy conservation that scales cumulatively with the number of samples processed. Second, within the Softsign pipeline, the realized energy savings are inversely proportional to the buffer capacity. If the FIFO size is exceptionally large, the MCU host naturally triggers fewer MCU wake-ups, which ultimately negates the energy-saving advantages of the 4-bit approach. However, it is crucial to note that FIFO buffers are primarily designed to mitigate sampling jitter. Deploying excessively large buffers to minimize wake-ups inherently delays data processing, severely compromising the real-time responsiveness of wearable systems. Consequently, practical implementations typically restrict FIFO capacities to a minimal range (e.g., 10 to 100 bytes) to satisfy strict latency constraints. Operating strictly within this realistic hardware envelope, the proposed 4-bit Softsign pipeline guarantees sustained and significant energy efficiency over continuous execution.

6 Conclusions

Crucially, our evaluation demonstrates that extreme low-bit compression can be achieved without severely reducing predictive performance on different datasets including a variety of activities. Furthermore, our per-axis Softsign strategy exhibited high resilience to inter-subject variability, delivering an improvement in worst-case accuracy against outlier subjects when compared to the shared-axis approach.

Beyond predictive accuracy, our framework yields definitive energy advantages for realistic edge execution. Because the Softsign function leverages the hardware FPU on platforms like the Cortex-M4, it requires less CPU cycles per sample. In contrast, the γ-QUANT algorithm relies on software emulation for its power function, heavily inflating the cost to extremely high CPU cycles per sample. These results collectively establish the proposed Softsign 4-bit quantization pipeline as an exceptionally practical and energy-positive solution for continuous, always-on execution in memory-constrained wearable architectures.

References

[1] Asma Abu-Samah, Dalilah Ghaffa, Nor Fadzilah Abdullah, Noorfazila Kamal, Rosdiadee Nordin, Jennifer C. Dela Cruz, Glenn V. Magwili, and Reginald Juan Mercado. 2025. Deployment of TinyML-Based Stress Classification Using Computational Constrained Health Wearable. *Electronics* 14, 4 (Feb. 2025), 687. doi:10.3390/electronics14040687

[2] Davide Anguita, Alessandro Ghio, Luca Oneto, Xavier Parra, and Jorge L Reyes-Ortiz. 2013. A Public Domain Dataset for Human Activity Recognition Using Smartphones. *Computational Intelligence* (2013).

[3] Fabrizio Maria Aymone and Danilo Pietro Pau. 2024. Benchmarking In-Sensor Machine Learning Computing: An Extension to the MLCommons-Tiny Suite. *Information* 15, 11 (Oct. 2024), 674. doi:10.3390/info15110674

[4] Sizhen Bian, Mengxi Liu, Vitor Fortes Rey, Daniel Geißler, and Paul Lukowicz. 2025. TinierHAR: Towards Ultra-Lightweight Deep Learning Models for Efficient Human Activity Recognition on Edge Devices. In *Proceedings of the 2025 ACM International Symposium on Wearable Computers*. ACM, Espoo Finland, 163–169. doi:10.1145/3715071.3750410

[5] Marius Bock, Hilde Kuehne, Kristof Van Laerhoven, and Michael Moeller. 2024. WEAR: An Outdoor Sports Dataset for Wearable and Egocentric Activity Recognition. *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 8, 4, Article 175 (Nov. 2024), 21 pages. doi:10.1145/3699776

[6] Andreas Bulling, Ulf Blanke, and Bernt Schiele. 2014. A tutorial on human activity recognition using body-worn inertial sensors. *Comput. Surveys* 46, 3 (Jan. 2014), 1–33. doi:10.1145/2499621

[7] Francesco Daghero, Alessio Burrello, Chen Xie, Marco Castellano, Luca Gandolfi, Andrea Calimera, Enrico Macii, Massimo Poncino, and Daniele Jahier Pagliari. 2022. Human Activity Recognition on Microcontrollers with Quantized and Adaptive Deep Neural Networks. *ACM Transactions on Embedded Computing Systems* 21, 4 (2022), 1–28. doi:10.1145/3542819

[8] Robert David, Jared Duke, Advait Jain, Vijay Janapa Reddi, Nat Jeffries, Jian Li, Nick Kreeger, Ian Nappier, Meghna Natraj, Shlomi Regev, Rocky Rhodes, Tiezhen Wang, and Pete Warden. 2021. TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems. arXiv:2010.08678 [cs.LG] https://arxiv.org/abs/2010.08678

[9] Shizhuo Deng, Jiaqi Chen, Da Teng, Chuangui Yang, Dongyue Chen, Tong Jia, and Hao Wang. 2024. LHAR: Lightweight Human Activity Recognition on Knowledge Distillation. *IEEE Journal of Biomedical and Health Informatics* 28, 11 (Nov. 2024), 6318–6328. doi:10.1109/JBHI.2023.3298932

[10] Maha S. Diab and Esther Rodriguez-Villegas. 2022. Embedded Machine Learning Using Microcontrollers in Wearable and Ambulatory Systems for Health and Care Applications: A Review. *IEEE Access* 10 (2022), 98450–98474. doi:10.1109/ACCESS.2022.3206782

[11] Tarek Elfouly and Ali Alouani. 2025. A Comprehensive Survey on Wearable Computing for Mental and Physical Health Monitoring. *Electronics* 14, 17 (Aug. 2025), 3443. doi:10.3390/electronics14173443

[12] Espruino. n.d.. Bangle.js 2 Technical Information. https://www.espruino.com/Bangle.js2. Accessed: Dec. 18, 2025.

[13] Mishal Fatima, Shashank Agnihotri, Marius Bock, Kanchana Vaishnavi Gandikota, Kristof Van Laerhoven, Michael Moeller, and Margret Keuper. 2025. -Quant: Towards Learnable Quantization for Low-bit Pattern Recognition. arXiv:2509.22448 (Sept. 2025). doi:10.48550/arXiv.2509.22448 arXiv:2509.22448 [cs].

[14] Neha Gaud, Maya Rathore, and Ugrasen Suman. 2024. MHCNLS-HAR: Multiheaded CNN-LSTM-Based Human Activity Recognition Leveraging a Novel Wearable Edge Device for Elderly Health Care. *IEEE Sensors Journal* 24, 21 (Nov. 2024), 35394–35405. doi:10.1109/JSEN.2024.3450499

[15] Mohamed Imed Eddine Ghebriout, Halima Bouzidi, Halima Bouzidi, Smail Niar, Smail Niar, Hamza Ouarnoughi, and Hamza Ouarnoughi. 2024. Harmonic-NAS: Hardware-Aware Multimodal Neural Architecture Search on Resource-constrained Devices. (2024), 374–389.

[16] Shubham Gupta, Dr. Sweta Jain, Bholanath Roy, and Abhishek Deb. 2022. A TinyML Approach to Human Activity Recognition. *Journal of Physics: Conference Series* 2273, 1 (May 2022), 012025. doi:10.1088/1742-6596/2273/1/012025

[17] Shawn Hymel, Colby Banbury, Daniel Situnayake, Alex Elium, Carl Ward, Mat Kelcey, Mathijs Baaijens, Mateusz Majchrzycki, Jenny Plunkett, David Tischler, Alessandro Grande, Louis Moreau, Dmitry Maslov, Artie Beavis, Jan Jongboom, and Vijay Janapa Reddi. 2023. Edge Impulse: An MLOps Platform for Tiny Machine Learning. arXiv:2212.03332 [cs.DC] https://arxiv.org/abs/2212.03332

[18] Tobias King, Yexu Zhou, Tobias Röddiger, and Michael Beigl. 2025. MicroNAS for memory and latency constrained hardware aware neural architecture search in time series classification on microcontrollers. *Scientific Reports* 15, 1 (March 2025), 7575. doi:10.1038/s41598-025-90764-z

[19] Navita Kumari, Amulya Yadagani, Basudeba Behera, Vijay Bhaskar Semwal, and Somya Mohanty. 2024. Human motion activity recognition and pattern analysis using compressed deep neural networks. *Computer Methods in Biomechanics and Biomedical Engineering: Imaging Visualization* 12, 1 (Dec. 2024), 2331052. doi:10.1080/21681163.2024.2331052

[20] Ismail Lamaakal, Chaymae Yahyati, Yassine Maleh, Khalid El Makkaoui, Ibrahim Ouahbi, Ahmed A. Abd El-Latif, Mariam Zomorodi, and Basma Abd El-Rahiem. 2025. A tiny inertial transformer for human activity recognition via multimodal knowledge distillation and explainable AI. *Scientific Reports* 15, 1 (Nov. 2025), 42335. doi:10.1038/s41598-025-26297-2

[21] Edgar Liberis and Nicholas D. Lane. 2022. Differentiable Neural Network Pruning to Enable Smart Applications on Microcontrollers. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 6, 4 (Dec. 2022), 1–19. doi:10.1145/3569468

[22] Michele Magno, Xiaying Wang, Manuel Eggimann, Lukas Cavigelli, and Luca Benini. 2020. InfiniWolf: Energy Efficient Smart Bracelet for Edge Computing with Dual Source Energy Harvesting. In *2020 Design, Automation Test in Europe Conference Exhibition (DATE)*. 342–345. doi:10.23919/DATE48585.2020.9116218 arXiv:2003.00041 [eess].

[23] Francisco Ordóñez and Daniel Roggen. 2016. Deep Convolutional and LSTM Recurrent Neural Networks for Multimodal Wearable Activity Recognition. *Sensors* 16, 1 (Jan. 2016), 115. doi:10.3390/s16010115

[24] Radu Pascale, Dan Tudose, and Tatiana Ţurcanu. 2024. Open Hardware and Software Smartwatch. In *2024 23rd RoEduNet Conference: Networking in Education and Research (RoEduNet)*. IEEE, Bucharest, Romania, 1–5. doi:10.1109/RoEduNet64292.2024.10722548

[25] Waleed Bin Qaim, Aleksandr Ometov, Antonella Molinaro, Ilaria Lener, Claudia Campolo, Elena Simona Lohan, and Jari Nurmi. 2020. Towards Energy Efficiency in the Internet of Wearable Things: A Systematic Review. *IEEE Access* 8 (2020), 175412–175435. doi:10.1109/ACCESS.2020.3025270

[26] Anguita Davide Ghio Alessandro Oneto Luca Reyes-Ortiz, Jorge and Xavier Parra. 2013. Human Activity Recognition Using Smartphones. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C54S4K.

[27] Bidyut Saha, Riya Samanta, Soumya K. Ghosh, and Ram Babu Roy. 2024. TinyTNAS: GPU-Free, Time-Bound, Hardware-Aware Neural Architecture Search for TinyML Time Series Classification. arXiv:2408.16535 (Aug. 2024). doi:10.48550/arXiv.2408.16535 arXiv:2408.16535 [cs].

[28] Swapnil Sayan Saha, Sandeep Singh Sandha, and Mani Srivastava. 2022. Machine Learning for Microcontroller-Class Hardware: A Review. *IEEE Sensors Journal* 22, 22 (Nov. 2022), 21362–21390. doi:10.1109/JSEN.2022.3210773

[29] Dimitrios Samakovlis, Stefano Albini, Rubén Rodríguez Álvarez, Denisa-Andreea Constantinescu, Pasquale Davide Schiavone, Miguel Peón-Quirós, and David Atienza. 2024. Enabling Efficient Wearables: An Analysis of Low-Power Microcontrollers for Biomedical Applications. arXiv:2411.09534 (Nov. 2024). doi:10.48550/arXiv.2411.09534 arXiv:2411.09534 [eess].

[30] Vinamra Sharma, Danilo Pau, and José Cano. 2024. Efficient Tiny Machine Learning for Human Activity Recognition on Low-Power Edge Devices. In *2024 IEEE 8th Forum on Research and Technologies for Society and Industry Innovation (RTSI)*. IEEE, Milano, Italy, 85–90. doi:10.1109/RTSI61910.2024.10761203

[31] Nazli Tekin, Ahmet Aris, Abbas Acar, Selcuk Uluagac, and Vehbi Cagri Gungor. 2024. A review of on-device machine learning for IoT: An energy perspective. *Ad Hoc Networks* 153 (Feb. 2024), 103348. doi:10.1016/j.adhoc.2023.103348

[32] Manny Villa and Eduardo Casilari. 2025. The Impact of the Accelerometer Sampling Rate on the Performance of Machine and Deep Learning Models in Wearable Fall-Detection Systems. *Sensors* 26, 1 (Dec. 2025), 162. doi:10.3390/s26010162

[33] Lars Wulfert, Johannes Kühnel, Lukas Krupp, Justus Viga, Christian Wiede, Pierre Gembaczka, and Anton Grabmaier. 2024. AIfES: A Next-Generation Edge AI Framework. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46, 6 (2024), 4519–4533. doi:10.1109/TPAMI.2024.3355495

[34] Jian Bo Yang, Minh Nhut Nguyen, Phyo Phyo San, Xiao Li Li, and Shonali Krishnaswamy. 2015. Deep convolutional neural networks on multichannel time series for human activity recognition. In *Proceedings of the 24th International Conference on Artificial Intelligence* (Buenos Aires, Argentina) *(IJCAI'15)*. AAAI Press, 3995–4001.

[35] Haotian Zhou, Xiujun Zhang, Yu Feng, Tongda Zhang, and Lijuan Xiong. 2025. Efficient human activity recognition on edge devices using DeepConv LSTM architectures. *Scientific Reports* 15, 1 (April 2025), 13830. doi:10.1038/s41598-025-98571-2

[36] Yexu Zhou, Tobias King, Haibin Zhao, Yiran Huang, Till Riedel, and Michael Beigl. 2024. MLP-HAR: Boosting Performance and Efficiency of HAR Models on Edge Devices with Purely Fully Connected Layers. In *Proceedings of the 2024 ACM International Symposium on Wearable Computers*. ACM, Melbourne VIC Australia, 133–139. doi:10.1145/3675095.3676624

[37] Yexu Zhou, Haibin Zhao, Yiran Huang, Till Riedel, Michael Hefenbrock, and Michael Beigl. 2022. TinyHAR: A Lightweight Deep Learning Model Designed for Human Activity Recognition. *Proceedings - International Symposium on Wearable Computers, ISWC* (2022), 89 – 93. doi:10.1145/3544794.3558467

[38] Minglu Zhu, Shuhan He, Tao Chen, and Chengkuo Lee. 2025. Wearable Intelligent Human–Machine Interfaces Ready for Sustainable Edge Computing Systems. *AI Sensors* 1, 2 (Dec. 2025), 9. doi:10.3390/aisens1020009

A Appendix: Implementation and Cycle Cost Analysis

This appendix provides the low-level algorithmic implementations for the data processing stages evaluated in this study. The instruction-level breakdown of these algorithms directly derives the CPU cycle costs utilized in our theoretical energy efficiency model, which are summarized in Table 7.

First of all, regarding the data packing stage($C_{packing}$), the proposed 4-bit pipeline efficiently packs two 4-bit samples into a single byte using standard bitwise operations. As detailed in Algorithm 1, this requires only 3 CPU cycles for two samples. Conversely, Algorithm 3 demonstrates that the 12-bit baseline must distribute two 12-bit samples across three distinct bytes, incurring a higher cost of 9 cycles. Next, for both pipelines, the number of CPU cycles for transitioning the MCU from a sleep state to an active state (C_{wake}) is a constant 1056 cycles per wake-up event. To unpack the signals prior to inference ($C_{unpacking}$), Algorithm 2 extracts two 4-bit samples from a single byte in just 3 cycles. Algorithm 4 requires 7 cycles to extract and reconstruct two 12-bit samples from a three-byte payload. The data movement operations ($C_{move(sensor)}$ or $C_{move(MCU)}$) are proportionally reduced. Algorithm 6 illustrates that moving two packed 4-bit samples requires writing only 1 byte, taking 2 cycles. Moving two 12-bit samples as Algorithm 5 requires looping over 3 bytes, costing 6 cycles.

The quantization stage ($C_{quantization}$) introduces a computational step that depends heavily on the chosen mathematical function. Algorithm 7 defines the Softsign-QUANT implementation, which leverages the Cortex-M4's hardware FPU to execute in 28 cycles per sample. In contrast, Algorithm 8 outlines the γ-QUANT method; because the power function is not natively supported by the FPU, it requires software emulation, driving the cost up to approximately 153 cycles per sample.

Algorithm 1: Pack two 4-bit samples into 1 byte

Input: $t0$ = sample1 (4-bit), $t1$ = sample2 (4-bit), $a0$ = FIFO base address
Output: One byte written to address $a0$

```
t2 ← (t0 ≪ 4)                          // move sample1 to upper nibble
t2 ← t2 OR (t1 & 0x0F)                 // merge sample2 into lower nibble
StoreByte(a0, t2)                      // store packed byte
```

Algorithm 2: Unpack 1 byte into 2 4-bit samples

Input: $a0$ = data buffer address
Output: $t0$ = sample1 (4-bit), $t1$ = sample2 (4-bit)

```
t2 ← LoadByte(a0)                      // load packed byte
t0 ← (t2 & 0x0F)                       // extract lower nibble
t1 ← (t2 ≫ 4)                          // extract upper nibble
```

Received 20 February 2026; revised 12 March 2026; accepted 12 March 2026

Algorithm 3: Pack two 12-bit samples into 3 bytes

Input: $t0$ = sample1 (12-bit), $t1$ = sample2 (12-bit), $a0$ = FIFO base address
Output: Three bytes written to addresses $a0, a0 + 1, a0 + 2$

```
t2 ← (t0 ≫ 4)                          // top 8 bits of sample1
StoreByte(a0, t2)                      // store byte 0
t3 ← (t0 & 0x0F)                       // bottom 4 bits of sample1
t3 ← (t3 ≪ 4)                          // move to upper nibble
t4 ← (t1 ≫ 8)                          // top 4 bits of sample2
t3 ← (t3 OR t4)                        // combine into byte 1
StoreByte(a0 + 1, t3)                  // store byte 1
t5 ← (t1 & 0xFF)                       // bottom 8 bits of sample2
StoreByte(a0 + 2, t5)                  // store byte 2
```

Algorithm 4: Unpacking 3 bytes and reconstruct 2 12-bit samples

Input: $a0$ = data buffer address
Output: $t4$ and $t5$ reconstructed from three consecutive bytes

```
t0 ← LoadByte(a0)                      // byte 0
t1 ← LoadByte(a0 + 1)                  // byte 1
t2 ← LoadByte(a0 + 2)                  // byte 2
t3 ← (t1 & 0x0F)                       // lower nibble of byte 1
t4 ← t0 OR (t3 ≪ 8)                    // reconstruct sample 1
t3 ← (t1 ≫ 4)                          // upper nibble of byte 1
t5 ← t3 OR (t2 ≪ 4)                    // reconstruct sample 2
```

Algorithm 5: Copy from Sensor FIFO to MCU SRAM (12-bit raw data, $N = 2$). Copy from Sensor Core SRAM to Sensor FIFO will be similar to the aforementioned.

Input: $a0$ = peripheral address, $a1$ = SRAM buffer address
Output: 3 bytes stored starting at $a1$

```
for t3 ← 0 to 2 do
    t1 ← LoadByte(a0)
    StoreByte(a1 + t3, t1)
end
```

Algorithm 6: Copy from Sensor FIFO to MCU SRAM (4-bit packed data, $N = 2$). Copy from Sensor Core SRAM to Sensor FIFO will be similar to the aforementioned.

Input: $a0$ = peripheral address, $a1$ = SRAM buffer address
Output: 1 packed byte stored at $a1$

```
t1 ← LoadByte(a0)                      // byte contains 2 samples
StoreByte(a1, t1)
```

Algorithm 7: Softsign Quantization

Input: Input vector $\mathbf{X}$ of length N, slope k, center μ, bit-width B
Output: Quantized output vector $\mathbf{Q}$

$S \leftarrow (2^B - 1)/2$
$i \leftarrow 0$
while $i < N$ **do**
  $v \leftarrow \mathbf{X}[i]$
  $z \leftarrow k \cdot (v - \mu)$
  $d \leftarrow 1.0 + |z|;$
  $y_{raw} \leftarrow z/d;$ `// 14 cycles (ARM VDIV.F32)`
  $y \leftarrow y_{raw} \cdot S$
  $\mathbf{Q}[i] \leftarrow Clamp(\text{round}(y), B)$ `// Hardware clamp`
  $i \leftarrow i + 1$
end
return $\mathbf{Q}$

Table 5. CPU cycle cost per sample for the Softsign function. All operations used to calculate the softsign function are directly supported by the hardware FPU unit in the Cortex M4.

Instruction	**CPU cycle cost per sample**	**Comments**
$V \leftarrow X[i]$	1 + 1 = 2	Load a 12-bit number from register
$z \leftarrow k \cdot (v - \mu)$	1 + 1 = 2	Multiplication & Subtraction
$d \leftarrow 1.0 + \lvert z \rvert$	1 + 1 = 2	Abs & Sum
$y_{raw} \leftarrow z/d$	14	Division
$y \leftarrow y_{raw} \cdot S$	1	
$q \leftarrow \text{Clamp}(\text{round}(y), B)$	1 + 1 = 2	Rounding & Clamp
$Q[i] \leftarrow q$	2	Store
Loop control (++, $i < N$)	3	Loop control related
Total	**28**	

Algorithm 8: Gamma Quantization

Input: Input vector $\mathbf{X}$ of length N, exponent γ, center μ, bit-width B
Output: Quantized output vector $\mathbf{Q}$

$S \leftarrow (2^B - 1)/2$
$i \leftarrow 0$
while $i < N$ **do**
  $v \leftarrow \mathbf{X}[i]$
  $z \leftarrow v - \mu$
  $m \leftarrow |z|$
  $s \leftarrow \text{sign}(z)$
  $p \leftarrow \text{pow}(m, \gamma)$
  $y_{raw} \leftarrow s \cdot p$
  $y \leftarrow y_{raw} \cdot S$
  $\mathbf{Q}[i] \leftarrow Clamp(\text{round}(y), B)$ `// Hardware clamp`
  $i \leftarrow i + 1$
end
return $\mathbf{Q}$

Table 6. CPU cycle cost per sample for the gamma-quantization function. The power (pow) operation is not supported by the hardware FPU unit in the Cortex-M architecture, necessitating software emulation. Cycle counts depend on the specific library implementation; here, the SEGGER floating-point library is used as a baseline, where $x^y = \exp(y \cdot \ln(x)) = C_{exp} + C_{logarithm} \approx 40 + 100 = 140$ cycles.

Instruction	**CPU cycle cost per sample**	**Comments**
$V \leftarrow X[i]$	$1 + 1 = 2$	Load a 12-bit number from register
$z \leftarrow (v - \mu)$	1	Subtraction
$s \leftarrow \text{sign}(z)$	1	
$p \leftarrow \text{pow}(z, \gamma)$	140	Pow operation
$y \leftarrow s \cdot p$	1	
$y \leftarrow y_{raw} \cdot S$	1	
$q \leftarrow \text{Clamp}(\text{round}(y), B)$	$1 + 1 = 2$	Rounding & Clamp
$Q[i] \leftarrow q$	2	Store
Loop control (++, $i < N$)	3	Loop control related
Total	≈ 153	

Table 7. Comparison of CPU cycle costs for data processing operations between the standard 12-bit and proposed 4-bit pipelines.

Operation	**12-bit**	**4-bit**
$C_{\text{quantization}}$	0	Algorithm dependent (28 or 153 cycles per sample)
C_{packing}	9 cycles for 2 samples	3 cycles for 2 samples
C_{wake}	1056 cycles per wake-up	1056 cycles per wake-up
$C_{\text{unpacking}}$	7 cycles for 2 samples	3 cycles for 2 samples
$C_{\text{move(sensor)}}$ or $C_{\text{move(MCU)}}$	6 cycles for 2 samples	2 cycles for 2 samples